Hindi Next Word Prediction Using LLM with LoRA Or Q-LoRA Fine-Tuning

A thesis (Phase-II) submitted in partial fulfillment of the requirements for
the award of the degree of

MASTER OF TECHNOLOGY

in

DATA ANALYTICS

By

MAYANK

(205224010)

DEPARTMENT OF COMPUTER APPLICATIONS
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI – 620015

JUNE 2026

BONAFIDE CERTIFICATE

This is to certify that the project titled **Hindi Next Word Prediction Using LLM with LoRA Or Q-LoRA Fine-Tuning** is a bonafide record of the work done by

MAYANK

(205224010)

in partial fulfillment of the requirements for the award of the degree of **Master of Technology** in **Data Analytics** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI**, during the year 2025–26.

Dr.(Mrs.) S. SANGEETHA
Guide

Dr. S. DOMNIC
Head of the Department

Project Viva-voce held on _____

Internal Examiner

External Examiner

ABSTRACT

The rapid advancement of Natural Language Processing (NLP) and Large Language Models (LLMs) has enabled significant progress in text generation and language understanding tasks. However, most state-of-the-art LLMs are primarily optimized for high-resource languages such as English, resulting in limited performance for low-resource and morphologically rich Indian languages like Hindi. This creates a barrier in developing intelligent systems such as next-word prediction, text generation, and conversational agents for regional languages. Traditional language models require large-scale labeled datasets and computational resources, which further restricts their applicability in resource-constrained environments.

This thesis presents an end-to-end system for Hindi next-word prediction using Large Language Models, addressing these challenges through efficient fine-tuning and data-driven approaches. First, a large-scale Hindi text dataset is constructed using Wikipedia and other publicly available sources, followed by extensive data cleaning and preprocessing to ensure high-quality input. Second, a tokenizer is developed to effectively handle Hindi text representation, enabling the model to learn meaningful linguistic patterns. Third, a transformer-based causal language modeling framework is implemented for next-word prediction, leveraging both custom-trained and pretrained LLM architectures.

To improve efficiency and reduce computational cost, parameter-efficient fine-tuning techniques, including Low-Rank Adaptation (LoRA) and Quantized LoRA (QLoRA), are applied. These approaches significantly reduce memory usage while maintaining competitive performance. Experimental results demonstrate that the proposed system achieves strong performance with low validation loss and perplexity, indicating effective learning of Hindi language structure. The model is further integrated into an interactive Streamlit-based user interface, enabling real-time next-word prediction and text generation.

By enabling accurate and efficient next-word prediction for Hindi, this system contributes toward bridging the gap in NLP capabilities for Indian languages and supports the development of intelligent applications such as chatbots, predictive typing systems, and language learning tools.

Keywords: Large Language Models, Hindi NLP, Next-Word Prediction, Causal Language Modeling, LoRA, QLoRA, Fine-tuning, Text Generation, Natural Language Processing.

ACKNOWLEDGEMENT

I would like to extend my gratitude to **Dr. G. AGHILA**, Director, National Institute of Technology, Tiruchirappalli for having provided all the facilities to carry out this project work.

I would like to thank **Dr. S. DOMNIC**, Head of the Department, Department of Computer Applications, National Institute of Technology, Tiruchirappalli for allowing me to avail the facilities of the department during project work.

I express my sincere thanks and gratitude to my guide **Dr.(Mrs.) S. SANGEETHA**, Associate Professor, Department of Computer Applications, National Institute of Technology, Tiruchirappalli who directed me from time to time throughout the project with valuable guidance, constant encouragement, and suggestions.

My hearty thanks to all the members of the Project Coordination Committee, Department of Computer Applications, National Institute of Technology, Tiruchirappalli, for their valuable suggestions during the project reviews.

I am also thankful to the faculty, staff members, and research scholars of the Department of Computer Applications, National Institute of Technology, Tiruchirappalli, for their support and suggestions throughout the project.

Mayank.

MAYANK

(205224010)

Contents

Title **Page**

ABSTRACT i

ACKNOWLEDGEMENT ii

TABLE OF CONTENTS iii

LIST OF TABLES vi

LIST OF FIGURES vii

LIST OF ABBREVIATIONS viii

CHAPTER 1 INTRODUCTION 1

1.1 General Introduction 1

1.2 Next-Word Prediction and Language Modeling 3

1.3 Motivation and Problem Context 5

1.4 Problem Statement 6

1.5 Objectives and Contributions 6

1.6 Structure of the Thesis 7

CHAPTER 2 LITERATURE REVIEW 9

2.1 Introduction 9

2.2 Transformer Architecture 9

2.3 Decoder-Only Transformer Models 10

2.4 How LoRA Fine-Tuning Works 11

2.5 How QLoRA Works 13

2.6 Challenges in Hindi Language Processing 14

2.7 Applications of Next-Word Prediction 15

CHAPTER 3 DATASET AND METHODOLOGY 16
3.1 Introduction 16
3.2 Dataset Description 17
3.2.1 Dataset Source 17
3.2.2 Dataset Preprocessing 17
3.3 Tokenizer Training 18
3.4 Model Selection 19
3.4.1 GPT-2 Trained from Scratch 20
3.4.2 GPT-2 with LoRA Fine-Tuning 20
3.5 LoRA Fine-Tuning Methodology 21
3.6 Q-LoRA Fine-Tuning Methodology 23
3.6.1 GPT-Neo-125M with LoRA Fine-Tuning 25
3.7 Training Configuration 25
3.8 Evaluation Metrics 26
3.8.1 Loss 26
3.8.2 Perplexity 27
3.9 Experimental Results 27
3.10 Inference (Next Word Prediction) 28
3.11 Deployment using Streamlit 29
CHAPTER 4 EXPERIMENTS AND RESULTS 30
4.1 Introduction 30
4.2 Experimental Setup 30
4.3 Training Configuration 31
4.4 Evaluation Metrics 32
4.4.1 Loss 32
4.4.2 Perplexity 33
4.5 LLM's Model Comparison 35
4.6 Inference Results 36
4.7 Qualitative Analysis of Generated Predictions 38

CHAPTER 5 CONCLUSION AND FUTURE WORK 39

5.1 Conclusion 39

5.2 Limitations 41

5.3 Future Work 42

REFERENCES 44

List of Tables

1.1 Example of next-word prediction in Hindi language modeling. . 4

3.1 Model Performance Comparison 27

4.1 Training Hyperparameters 31

4.2 Model Performance Comparison 35

List of Figures

1.1 LLM-based Next Word Prediction Pipeline 2

3.1 Sample representation of the Hindi dataset used for training . 18
3.2 Tokenization process using SentencePiece for Hindi text 19
3.3 Architecture of GPT-2 trained from scratch 20
3.4 GPT-2 with LoRA fine-tuning 21
3.5 GPT-Neo model with LoRA fine-tuning 25
3.6 Model performance comparison based on loss and perplexity . . 27
3.7 Example of next-word prediction using trained model 29

4.1 Training and validation loss comparison across models 34
4.2 Perplexity comparison across different models 34
4.3 Comparison of model performance based on loss and perplexity 36
4.4 Example of Hindi next-word prediction 37

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
BPE	Byte Pair Encoding
DL	Deep Learning
FFN	Feed-Forward Network
GPU	Graphics Processing Unit
GPT	Generative Pre-trained Transformer
LLM	Large Language Model
LM	Language Model
LoRA	Low-Rank Adaptation
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantized Low-Rank Adaptation
RNN	Recurrent Neural Network
LSTM	Long Short-Term Memory
SPM	SentencePiece Model
TF	TensorFlow
PT	PyTorch
UNK	Unknown Token
EOS	End of Sentence
BOS	Beginning of Sentence
VRAM	Video Random Access Memory

CHAPTER 1

INTRODUCTION

1.1 General Introduction

The rapid advancement of LLMs and NLP has transformed the way machines understand and generate human language. Transformer-based architectures have significantly enhanced applications such as machine translation, text summarization, conversational agents, and predictive text generation systems [1]. Among these applications, next-word prediction plays an essential role in enabling intelligent typing systems, chatbots, autocomplete systems, and AI-driven content generation platforms.

Over the past few years, NLP has emerged as one of the most active research domains within Artificial Intelligence. It enables machines to analyze, interpret, and generate human language in a meaningful and context-aware manner. Earlier NLP techniques mainly depended on rule-based systems and traditional statistical approaches, which often struggled to capture contextual meaning and semantic relationships. The emergence of deep learning and transformer-based architectures has greatly improved the ability of machines to process language efficiently. These modern models can learn contextual semantics, long-term dependencies, and grammatical structures directly from large-scale textual corpora.

Large Language Models are advanced neural network architectures trained on massive amounts of textual data in order to generate human-like text. These models are capable of performing several NLP tasks with remarkable efficiency. Modern applications such as intelligent virtual assistants, recommendation systems, search engines, automated text generation platforms, and conversational AI systems largely rely on language modeling techniques.

Despite these advancements, most state-of-the-art LLMs are primarily trained on high-resource languages such as English. Consequently, their performance often degrades when applied to low-resource and morphologically rich Indian languages like Hindi [2]. Hindi contains complex grammatical patterns, contextual variations, and diverse vocabulary, which makes accurate next-word prediction a challenging task. This limitation creates a considerable gap in the development of efficient NLP systems for regional Indian languages.

Hindi is among the most widely spoken languages globally and serves as a

major medium of communication for millions of users. However, compared to English, relatively fewer high-quality pretrained language models are available for Hindi. Processing Hindi text presents additional challenges such as free word order, inflectional forms, compound word construction, gender-specific grammar, and diverse writing styles. Due to these linguistic complexities, language models primarily trained on English data often fail to generate contextually accurate Hindi text.

Figure 1.1: Overview of the Hindi next-word prediction system, showing data preprocessing, tokenizer training, model training, and inference stages.

The introduction of transformer-based LLMs such as GPT [3], GPT-3 [4], and GPT-Neo [5] has created new possibilities for multilingual language processing tasks. These models can learn contextual relationships between words and generate coherent textual outputs. Nevertheless, applying pretrained LLMs directly to Hindi next-word prediction often produces suboptimal results because Hindi language data is comparatively underrepresented in the pretraining datasets.

To address these limitations, recent research has focused on adapting pretrained transformer models for regional languages using fine-tuning approaches. Instead of training large-scale models entirely from scratch, pretrained models can be efficiently adapted for downstream tasks using parameter-efficient fine-tuning techniques. Such approaches reduce computational requirements while still achieving competitive performance. In this thesis, LoRA and QLoRA techniques are employed to enhance Hindi next-word prediction while minimizing GPU memory consumption and training complexity.

The proposed work aims to develop an efficient Hindi next-word prediction

pipeline that integrates data preprocessing, tokenization, transformer-based causal language modeling, parameter-efficient fine-tuning, and evaluation using standard language modeling metrics. The overall workflow of the proposed system is illustrated in Fig. 1.1.

1.2 Next-Word Prediction and Language Modeling

Next-word prediction is one of the fundamental tasks in language modeling, where the objective is to determine the most probable word that follows a given sequence of words. Causal Language Modeling (CLM) is commonly employed for this purpose, where the model predicts the next token using previously observed tokens in the sequence [3].

Language models form the backbone of many modern NLP applications. These models estimate the probability distribution of a sequence of words and attempt to predict the next most likely word based on contextual information available from preceding words.

For example, consider the Hindi sentence:

भारत की राजधानी

The expected prediction would be:

दिल्ली

Similarly, for the sentence:

मुझे हिंदी

the model may predict:

सीखना

Such predictions are generated from probability distributions learned during the training phase.

Traditional language modeling approaches such as n-gram models face difficulty in capturing long-range contextual dependencies. Transformer-based architectures overcome this limitation using the self-attention mechanism, which enables the model to capture contextual relationships across the entire sequence [1].

Conventional statistical methods including unigram, bigram, and trigram models rely only on a limited context window for prediction. As a result, they often fail to capture semantic meaning and long contextual dependencies

in lengthy sentences. Transformer architectures address this problem through self-attention, where every token in the sequence can attend to all other tokens. This enables the model to learn contextual relationships more effectively and generate more accurate predictions.

Tokenization is another crucial component in Hindi language modeling. Due to the rich morphology and large vocabulary of Hindi, subword tokenization methods such as SentencePiece [6] and Byte Pair Encoding (BPE) [7] are widely used. Proper tokenization enables the model to learn meaningful representations of words and subwords, thereby improving prediction quality.

Hindi contains numerous word variations arising from prefixes, suffixes, grammatical transformations, and inflectional forms. Direct word-level tokenization may lead to extremely large vocabularies and higher computational complexity. Subword tokenization solves this issue by decomposing rare or complex words into smaller meaningful units, improving both model generalization and memory efficiency.

For example:

विद्यालयों

can be divided into subword tokens as:

विद्यालय + ों

This helps the model understand unseen or rare words more effectively during inference.

Table 1.1: Example of next-word prediction in Hindi language modeling.

Input Sequence	**Predicted Next Word**
भारत एक	महान
मुझे हिंदी	सीखना
वह स्कूल	जाता
आज मौसम	अच्छा

The effectiveness of next-word prediction systems is generally evaluated using metrics such as validation loss and perplexity. Perplexity is one of the most widely used evaluation metrics for language models because it measures how well the model predicts the next token in a sequence. Lower perplexity values indicate better contextual understanding and stronger prediction capability.

When a model predicts the next word with higher confidence and assigns greater probability to the correct token, the perplexity score decreases. Conversely,

when the model is uncertain and distributes probability across multiple possible tokens, the perplexity score increases. Therefore, minimizing perplexity is considered an important objective during language model training.

1.3 Motivation and Problem Context

The motivation behind this work arises from the growing need for advanced language modeling systems designed specifically for Indian languages. Although transformer-based models trained on English datasets achieve impressive performance, their effectiveness decreases considerably when applied to Hindi because of limited Hindi training resources and linguistic complexities [2].

The widespread adoption of digital communication platforms, educational technologies, and AI-based content generation tools has increased the demand for language technologies in regional languages. Millions of Hindi-speaking users interact daily with smartphones, computers, and online platforms. However, compared to English, the availability of advanced Hindi text generation systems remains limited.

Another significant challenge is the enormous computational cost associated with training large language models from scratch. Developing transformer-based models requires large-scale datasets, high-end GPUs, extensive training time, and substantial memory resources. These requirements are often difficult to fulfill in academic and resource-constrained environments. Consequently, parameter-efficient fine-tuning techniques have gained substantial attention in recent years.

In LoRA and QLoRA, only a small subset of trainable adapter parameters is updated while the original pretrained model parameters remain frozen. This drastically reduces the number of trainable parameters and lowers computational requirements. QLoRA further improves memory efficiency through low-bit quantization techniques. These methods allow researchers to fine-tune large language models without depending on expensive cloud-based GPU infrastructure.

Another important motivation for selecting this research topic is the limited availability of specialized Hindi next-word prediction systems based on transformer-based LLMs. Most existing Hindi NLP applications mainly focus on tasks such as translation or sentiment analysis, whereas research on Hindi text generation and next-word prediction remains comparatively underexplored.

1.4 Problem Statement

The primary research problem addressed in this thesis is stated as follows:

> *Using the fine-tuning technique such as LoRA and QLoRA to design an efficient large language model that accurately predicts the next word from Hindi text sequence with minimal processing cost and high accuracy.*

Several technical challenges must be addressed to solve this problem effectively:

- Developing a sufficiently large Hindi text corpus for model training.
- Designing an appropriate tokenization strategy for Hindi language processing.
- Fine-tuning pretrained transformer models for Hindi next-word prediction tasks.
- Applying parameter-efficient techniques such as LoRA and QLoRA to reduce computational cost.
- Evaluating model performance using metrics such as validation loss and perplexity.

Another important challenge involves maintaining a balance between computational efficiency and prediction accuracy. Transformer-based models with large parameter sizes generally provide better performance but require significantly higher GPU memory and training time. Therefore, this research explores parameter-efficient fine-tuning strategies to achieve effective Hindi language modeling with lower computational requirements.

1.5 Objectives and Contributions

The major objectives of this thesis are listed below:

- Collecting a large-scale Hindi text corpus from sources such as Wikipedia.
- Performing preprocessing and cleaning operations on the dataset for efficient model training.
- Applying tokenization techniques suitable for Hindi language modeling.
- Training and evaluating GPT-based transformer models for Hindi next-word prediction.

- Implementing LoRA and QLoRA for parameter-efficient fine-tuning.
- Evaluating model performance using validation loss and perplexity metrics.
- Comparing different language models including GPT-based architectures, GPT-2, and GPT-Neo 125M.
- Analyzing the impact of parameter-efficient fine-tuning on Hindi text generation performance.

The major contributions of this thesis include:

- Development of a Hindi next-word prediction system using transformer-based large language models.
- Application of LoRA and QLoRA for efficient parameter fine-tuning.
- Preparation and construction of a large Hindi language dataset for training purposes.
- Evaluation of model performance using validation loss and perplexity metrics.
- Comparative analysis between GPT models trained from scratch and pretrained transformer architectures.
- Investigation of transformer model adaptation for Hindi language generation tasks.
- Deployment of the trained model through a web-based interface for real-time prediction.

Furthermore, this research contributes to the broader area of low-resource language processing in LLMs by demonstrating that state-of-the-art parameter-efficient fine-tuning approaches can be successfully adapted for Hindi text generation. The findings of this work may support future research in Indian language NLP, conversational AI systems, educational applications, and intelligent chatbot development.

1.6 Structure of the Thesis

The remaining chapters of this thesis are organized as follows:

- **Chapter 2** presents the literature review related to language modeling, transformer architectures, and parameter-efficient fine-tuning methods such as LoRA and QLoRA.
- **Chapter 3** explains the methodology adopted in this research, including data collection, preprocessing, tokenization, transformer model architecture, and fine-tuning strategies.
- **Chapter 4** discusses the experimental setup, training configuration, evaluation metrics, perplexity computation, validation loss analysis, and comparative performance evaluation of GPT-based models, GPT-2, and GPT-Neo 125M.
- **Chapter 5** summarizes the conclusions of the thesis, highlights important contributions, discusses limitations, and provides directions for future research.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

The objective to next-word prediction in NLP is closely relates several important research issued,language modeling,text creation,multilingual learning,transformer topologies,and parameter-efficient fine-tuning strategies.Despite significant advancements in English language processing,resource limitations,linguistic complexity make it challenging to applied these technologies to Indian languages like Hindi [2]. Thanks to developments in DL and LLM's based topologies, NLP has expanded dramatically over previous decade.Foundation of language processing systems used to be rule-based approaches and statistical techniques like n-gram techniques and HMMs.Those techniques was effectived for straightforward tasks,they were unable to capture long-range semantic connections and contextual dependencies in language. NLP systems were greatly enhanced by neural network architectures, which allowed models to directly learn contextual representations from unprocessed textual data. Modern LLMs can generate contextually appropriate content by using autoregressive language modeling techniques. These advancements have substantially helped to applications,conversational Artifical Intelligence,machine translation,automated summarization, predictive typing, and question-answering systems.

2.2 Transformer Architecture

The most significant advancements in DL and NLP research is the LLM's diagrams,which was first presented by Vaswani et al. [1].
The self-attention mechanism, which allowed the LLM's model to recognize associations between words in a sequence independent, spatial distance,is the main mechanism used by transformers.
Attention mechanism computes three important vectors:

- Query (Q)
- Key (K)
- Value (V)

The attention operation is drived as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where:

- Q represented as query matrix
- K represented as key matrix
- V represented as the value matrix
- d_k is dimensionality to key vectors

For example :

भारत एक महान देश है

the transformer learns contextual relationships between:

- भारत
- महान
- देश

2.3 Decoder-Only Transformer Models

Autoregressive text generation jobs frequently employ decoder-only transformer designs.That category included to GPT-style models, which are made especially for next-token prediction. Decoder-only models produce text sequentially by predicting one character at a time,in contrast to encoder-decoder designs used in translation tasks.
The general definition of the prediction aimed is:

$$P(X) = \prod_{t=1}^{n} P(X_t|X_{<t}) \tag{2.2}$$

where:

- X_t represents the current token
- $X_{<t}$ represents all previous tokens

GPT-style architectures utilize masked self-attention to ensured that future tokens did't visible during prediction.
This autoregressive mechanism makes decoder-only transformers highly effective for:

- Next-word prediction
- Text generation
- Chatbot systems
- Predictive typing
- Conversational AI

2.4 How LoRA Fine-Tuning Works

A PEFT method called Low-Rank Adaptation (LoRA) was created to lessen the computational complexity involved in training LLM's models [8].
All pretrained model weights are modified during training in conventional transformer fine-tuning. Assume the following representation of a transformer weight matrix:

$$W \in \mathbb{R}^{d \times k} \tag{2.3}$$

Millions of or Even Billions of parameters must be updated in order to fully fine-tune the matrix W.
LoRA does not immediately update the entire weight matrix. Rather, two smaller low-rank matrices is used to approximate the weight update:

$$\Delta W = BA \tag{2.4}$$

where:

- $B \in \mathbb{R}^{d \times r}$
- $A \in \mathbb{R}^{r \times k}$
- r is the low-rank dimension

The updated transformer weight becomes:

$$W' = W + BA \tag{2.5}$$

Instead training,original large matrix W, LoRA trains only small matrices A and B.
For example:
Suppose:

$$W \in \mathbb{R}^{4096 \times 4096} \tag{2.6}$$

Full fine-tuning would require updating:

$$4096 \times 4096 = 16,777,216 \tag{2.7}$$

parameters.
If LoRA rank $r = 8$:

$$A \in \mathbb{R}^{8\times 4096} \tag{2.8}$$

$$B \in \mathbb{R}^{4096\times 8} \tag{2.9}$$

Total trainable parameters become:

$$(8 \times 4096) + (4096 \times 8) \tag{2.10}$$

$$= 65536 \tag{2.11}$$

which is dramatically smaller than full fine-tuning. This reduction provides several advantages:

- Lower GPU memory usage
- Faster training
- Reduced computational cost
- Efficient storage of adapters
- Easier deployment

LoRA techniques is applied to the transformer attention matrices such as:

- Query projection
- Key projection
- Value projection
- Output projection

Because lightweight matrice wasupdated,the original pretrained knowledge remains mostly preserved when adapting the model toward Hindi language understanding.

2.5 How QLoRA Works

Quantized Low-Rank Adaptation (QLoRA) extends LoRA by introducing low-bit quantization into pretrained transformer weights [9].
Large transformer models often require enormous GPU memory because weights are stored in high precision formats.
QLoRA reduces this memory requirement using 4-bit quantization.
In quantization:

$$W_{fp32} \rightarrow W_{int4} \tag{2.12}$$

The original floating-point weights were compressed into low-bit representations while maintaining most of the model's predictive capability.
QLoRA combines:

- 4-bit quantization
- Frozen pretrained weights
- LoRA adapter training

The quantized model weights remain fixed during training, while LoRA adapters are updated.
The memory reduction achieved by quantized could be approximated as:

$$MemoryReduction \approx \frac{32}{4} = 8\times \tag{2.13}$$

Thus,QLoRA can be reduce memory consumption of nearly eight times compared to FP32 training.
For example:
Suppose a model originally requires:

$$24GB \tag{2.14}$$

GPU memory using FP32 precision.
After 4-bit quantization:

$$\frac{24}{8} = 3GB \tag{2.15}$$

approximately.
This allows large transformer models can be fine-tuned even on consumer-grade GPUs.
QLoRA also introduces techniques like:

- Double Quantization
- NormalFloat (NF4) quantization
- Paged optimizers

2.6 Challenges in Hindi Language Processing

Hindi language modeling presented several challenges are compared to English NLP systems.
Some major challenges include:

- Rich morphology
- Flexible sentence structure
- Compound word formation
- Limited annotated datasets
- Scarcity of pretrained Hindi LLMs
- Computational limitations

Hindi words often contain multiple grammatical inflections based on:

- Gender
- Number
- Tense
- Context

For example:

जाता

and

जाती

represent gender-specific contextual variations.
Another challenge involves code-mixed Hindi-English text frequently used on social media platform's.
For examples:

आज की मीटिंग बहुत अच्छी थी।

Such mixed-language patterns increase tokenization and contextual learning complexity.

Therefore, effective Hindi NLP systems require robust tokenization, multilingual adaptation, and efficient transformer fine-tuning strategies.

2.7 Applications of Next-Word Prediction

Many contemporary NLP applications are built on next-word prediction.Text generation systems,intelligent chatbots,machine translation,conversational AI,predictive typing tools all make extensive use of it [4]. Estimating the most likely next token based on previously observed context is the goal of next-word prediction systems.Those systems are the foundation of contemporary generatived AI's applications,autoregressive language models.

Predictive typing the on smartphones and messaging apps is one of the most popular uses of next-word prediction.By recommending words and phrases that are contextually appropriate,these systems enhance typing speed and user experience. Machine translation, language models help improve fluency by selecting contextually appropriate words during sentence generation. Similarly,text summarization systems utilize language modeling techniques to generate coherent summaries from lengthy documents. For multilingual environments,next-word prediction systems can improve accessibility by supporting intelligent regional language applications.Hindi language generation systems may be contribute to educational tools,assistive technologies,conversational platforms designed for native speakers.

Applications like smart keyboards,AI-powered writing assistants, educational software,local search systems can particularly benefit from Hindi language generation technologies.Such systems could helped bridge the digital divide by enabling non-English-speaking communities to access AI-driven tools more effectively.

CHAPTER 3

DATASET AND METHODOLOGY

3.1 Introduction

The general approach Hindi next-word prediction using LLMs is presented. The dataset preparation procedure, preprocessing methods, tokenizer training, transformer model selection, parameter-efficient fine-tuning strategies, assessment methodologies, and deployment pipeline are all explained. The suggested system predicts the likely next word in a Hindi sentence using pretrained transformer architectures [1] and effective fine-tuning techniques like LoRA (Low-Rank Adaptation) [8] and QLoRA (Quantized Low-Rank Adaptation) [9]. The approach focuses creating scalable and computationally effective Hindi language modeling that produce next-word predictions that make sense in context. Dataset collection, preprocessing, tokenizer creation, transformer training, parameter-efficient adaptation, inference generation, deployment comprise the entire procedure. High-quality textual corpora and efficient preprocessing techniques are crucial for large language models. As result, lot work went into creating rich clean Hindi dataset that might enhance language representation learning and contextual comprehension. Additionally, suggested approach places a heavy emphasis on using LoRA and QLoRA fine-tuning strategies to reduce computational complexity without sacrificing prediction performance. The overall methodology includes:

- Data collection and corpus preparation
- Data cleaning and preprocessing
- Tokenizer training
- Transformer model selection
- Parameter-efficient fine-tuning
- Model evaluation using loss and perplexity
- Inference generation and deployment

3.2 Dataset Description

3.2.1 Dataset Source

This study makes a specially constructed corpus called: **Hindi Next Word Prediction Dataset** Effectiveness of transformer-based language models is largely dependent the caliber variety of training. The dataset was gathered several Hindi textual resources formal and semi-formal language patterns in order to enhance vocabulary coverage contextual comprehension. Dataset sources include:

- Public Hindi datasets [2]
- Online Hindi articles and blogs
- Hindi educational resources
- Hindi books and literature

Additional text samples was collected publicly available educational material, news-style content, and general Hindi literature to increase linguistic diversity. Combining multiple sources help model learn broader of grammatical structures, contextual relationships, sentence formations. The dataset contain nearly 167850 rows of Hindi textual sequences. Each row consists of meaningful Hindi sentences or phrases suitable for next-token prediction tasks. The dataset covers multiple domains including education, social topics, history, science, technology, and general conversational text.

3.2.2 Dataset Preprocessing

Noisy information like HTML tags, undesired symbols, duplicate sentences, uneven formatting, and unfinished text is frequently included in raw textual data gathered internet sources. Preprocessing becomes a crucial stage prior to language model training. following steps are part of the preprocessing pipeline utilized in work:

- Removal of null empty entries
- Elimination of duplicate textual samples
- Unicode normalization for Hindi characters
- Removal of unnecessary symbols and special characters
- Whitespace normalization

- Sentence cleaning and formatting

For example:

भारत!!! एक महान देश है....

After preprocessing:

भारत एक महान देश है

Duplicate sentence removal also helps reduce memorization and improves generalization capability. Similarly, Unicode normalization ensures Hindi characters is represented consistently throughout the dataset.

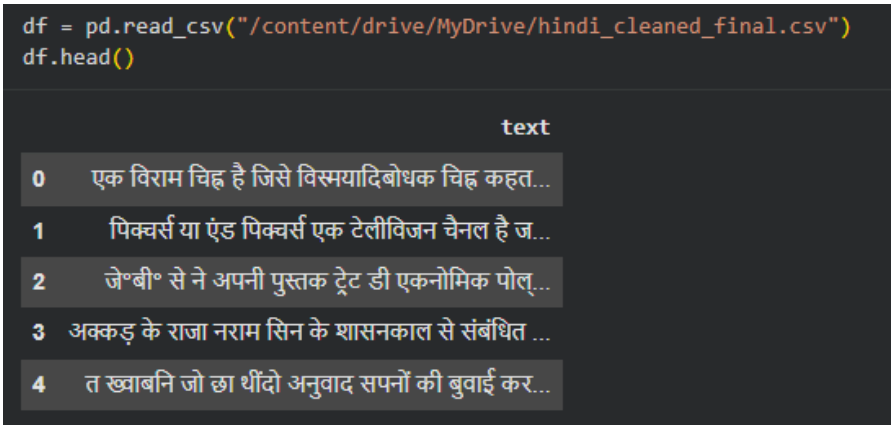

Figure 3.1: Sample representation of the Hindi dataset used for training

3.3 Tokenizer Training

For this study, SentencePiece tokenizer based Byte Pair Encoding (BPE) was trained [6, 7]. Since transformer systems is unable interpret raw textual input directly, tokenization is most crucial steps in the language modeling pipeline. Before neural language can comprehend input phrases, they must first be transformed into tokens and matching numerical token IDs. Hindi is a morphologically rich language with compound words, suffixes, prefixes, inflectional forms, and intricate grammatical structures. Extremely big vocabularies and more out-of-vocabulary issues might result from word-level tokenization. Subword tokenization methods is therefore more suited representing the Hindi language. Two popular subword tokenization techniques that split words into more manageable chunks are SentencePiece and Byte Pair Encoding (BPE).These methods help the model handle rare words and improve vocabulary efficiency. tokenizer was trained using the following configuration:

- Vocabulary Size: 8000

A vocabulary size of 8000 was selected to balance computational efficiency and semantic representation. Very small vocabularies may fail to capture meaningful contextual patterns, whereas excessively large vocabularies increase memory consumption and training complexity.

Example Sentence:

भारत एक महान देश है

Tokenized Output:

The tokenizer converts sentence in subword tokens corresponding token IDs that can processed by transformer model. For example:

भारत → Token ID: 145

महान → Token ID: 892

For Hindi, subword tokenization works especially well since many root terms have many grammatical forms. During inference, the model may more effectively generalize across unknown or uncommon language by learning subword-level representations. Vocabulary reduction is significant benefit of BPE-based tokenization. Subword units are created combining frequently occurring character pairings, which reduces vocabulary size without sacrificing semantic meaning.

```
Sample text : भारत एक महान देश है।
Tokens      : ['_भारत', '_एक', '_महान', '_देश', '_है', '।']
Token IDs   : [72, 50, 1637, 592, 15, 7960]
Decoded     : भारत एक महान देश है।
Vocab size  : 8000
```

Figure 3.2: Tokenization process using SentencePiece for Hindi text

3.4 Model Selection

Three transformer-based models were used in this study:

- **Model 1: GPT-2 (Trained from Scratch)** [3]
- **Model 2: GPT-2 (Pretrained + LoRA Fine-Tuning)** [8]
- **Model 3: GPT-Neo-125M (Pretrained + LoRA Fine-Tuning)** [5]

Using multiple transformer models allows comparison between training-from-scratch approaches and pretrained transfer learning strategies. Each model was selected to evaluate contextual learning capability, transfer learning performance, and parameter-efficient adaptation for Hindi next-word prediction.

3.4.1 GPT-2 Trained from Scratch

Using the unique Hindi dataset, first model was trained completely from scratch. method used Hindi textual data to directly train transformer parameters, which was initialized at random. model may acquire language-specific contextual patterns from given corpus when it is trained from scratch. Larger datasets, more processing power longer training times are often needed for this method, though. specialization toward Hindi language structures is one benefit of scratch training. However, scratch-trained models could find difficult to match the performance of big pretrained transformer architectures because to hardware limitations and small dataset sizes. Massive corpora are frequently necessary transformer models trained from scratch to efficiently learn semantic connections. Pretrained transfer learning techniques often offer superior contextual awareness in low-resource environments.

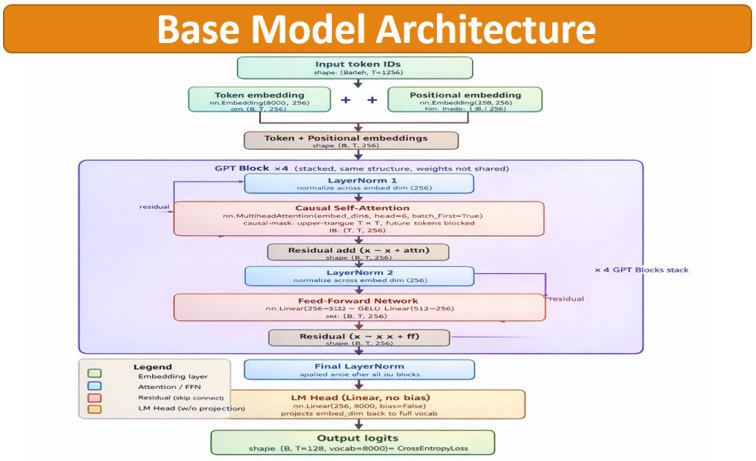

Figure 3.3: Architecture of GPT-2 trained from scratch

3.4.2 GPT-2 with LoRA Fine-Tuning

The second model uses LoRA-based fine-tuning in conjunction pretrained GPT-2 weights. LoRA adds trainable low-rank matrices to transformer attention layers rather updating all transformer parameters. method preserves the majority of pretrained information acquired during large-scale language modeling while drastically reducing the number of trainable parameters. So

its extensive training on textual datasets, pretrained GPT-2 already possesses robust contextual representations. model may be adjusted to Hindi syntax, vocabulary, contextual patterns by fine-tuning it using Hindi text. Reduced GPU memory use is one of main benefits of LoRA fine-tuning. Compared to comprehensive fine-tuning, the computational cost is significantly reduced because just adapter parameters are modified during training.

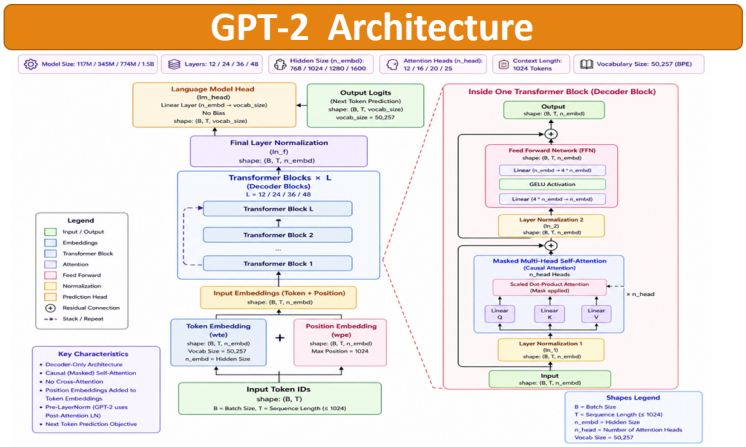

Figure 3.4: GPT-2 with LoRA fine-tuning

3.5 LoRA Fine-Tuning Methodology

A parameter-efficient fine-tuning method called Low-Rank Adaptation (LoRA) was created lower computational complexity of training large transformer models [8]. LoRA incorporates tiny trainable low-rank matrices transformer attention layers freezes the original model parameters rather updating all pretrained transformer weights. During training, millions or even billions of parameters are updated via traditional transformer fine-tuning. These methods need a lot of GPU memory, long training period, and lot of processing power. To solve this issue, LoRA maintains much of the pretrained information while training just lightweight adapter matrices. Assume a pretrained weight matrix is included transformer attention layer:

$$W \in \mathbb{R}^{d \times k}$$

During conventional fine-tuning, entire matrix W is updated. However, in LoRA, the weight update is approximated using two smaller low-rank matrices:

$$\Delta W = BA$$

where:

- $B \in \mathbb{R}^{d \times r}$
- $A \in \mathbb{R}^{r \times k}$
- $r \ll d, k$

The updated transformer weight becomes:

$$W' = W + BA$$

The rank r is intentionally kept small compared to original matrix dimensions. This reduces total number of trainable parameters significantly. For example, consider:

$$d = 768, \quad k = 768, \quad r = 8$$

Without LoRA:

$$768 \times 768 = 589824$$

trainable parameters are required.
With LoRA:

$$(768 \times 8) + (8 \times 768)$$

$$6144 + 6144 = 12288$$

trainable parameters is required. This demonstrates that LoRA dramatically reduces training complexity while maintaining performance. LoRA is commonly applied transformer attention matrices such as:

- Query projection matrix
- Key projection matrix
- Value projection matrix
- Output projection matrix

The LoRA training workflow used in research includes:

- Loading pretrained GPT-2 or GPT-Neo weights
- Freezing original transformer parameters

- Injecting LoRA adapter matrices
- Fine-tuning only adapter parameters
- Saving LoRA weights separately

For example:

Input: भारत एक

Expected Output:

महान

During fine-tuning, LoRA adapters learn contextual Hindi relationships while leveraging the pretrained transformer knowledge already present inside GPT-style architectures.

3.6 Q-LoRA Fine-Tuning Methodology

Quantized Low-Rank Adaptation (QLoRA) extends the LoRA approach by combining parameter-efficient fine-tuning with low-bit quantization [9]. QLoRA enables large transformer models to be fine-tuned using significantly lower GPU memory while preserving strong language modeling performance. primary objective of quantization is to reduce memory usage by storing pretrained model weights in compact numerical formats. using 16-bit or 32-bit floating-point representations, QLoRA commonly utilizes 4-bit quantized weights. In standard transformer training, model weights are stored using high-precision floating-point values:

$$W_{FP16}$$

QLoRA converts these weights into lower precision representations:

$$W_{4bit}$$

This dramatically reduces GPU memory requirements during training and inference. QLoRA framework combines:

- 4-bit quantized pretrained weights
- LoRA adapter matrices
- Frozen transformer backbone

- Parameter-efficient gradient updates

The overall weight update formulation becomes:

$$W' = W_{4bit} + BA$$

where:

- W_{4bit} represents quantized transformer weights
- BA represents LoRA adapter updates

Only the LoRA matrices are updated during training, while quantized transformer weights remain frozen. example, suppose a transformer model originally requires:

16 GB GPU Memory

After applying 4-bit quantization:

4 GB GPU Memory

may become sufficient for fine-tuning. QLoRA also introduces advanced optimization techniques such as:

- Double quantization
- NormalFloat (NF4) quantization
- Paged optimizers

NF4 quantization is specifically designed normally distributed neural network weights and helps preserve model accuracy despite aggressive compression. The QLoRA training pipeline includes:

- Loading pretrained transformer weights
- Applying 4-bit quantization
- Injecting LoRA adapters
- Freezing quantized backbone weights
- Fine-tuning lightweight LoRA matrices

The advantages of QLoRA include:

- Extremely low GPU memory usage
- Faster fine-tuning
- Reduced computational cost
- Efficient large-model adaptation
- Practical deployment on low-resource systems

3.6.1 GPT-Neo-125M with LoRA Fine-Tuning

The third model combines GPT-Neo-125M with LoRA adaptation. GPT-Neo is an open-source autoregressive transformer architecture designed for large-scale text generation tasks. Compared to smaller transformer architectures, GPT-Neo contains stronger pretrained contextual representations and larger transformer capacity. Fine-tuning GPT-Neo on Hindi data improves contextual understanding and next-word prediction quality. LoRA fine-tuning further reduces training complexity and GPU memory requirements, making large-model adaptation feasible even in resource-constrained research settings. Because GPT-Neo was pretrained on large textual corpora, it already possesses strong linguistic knowledge. Hindi-specific fine-tuning helps adapt these pretrained representations toward regional language generation tasks.

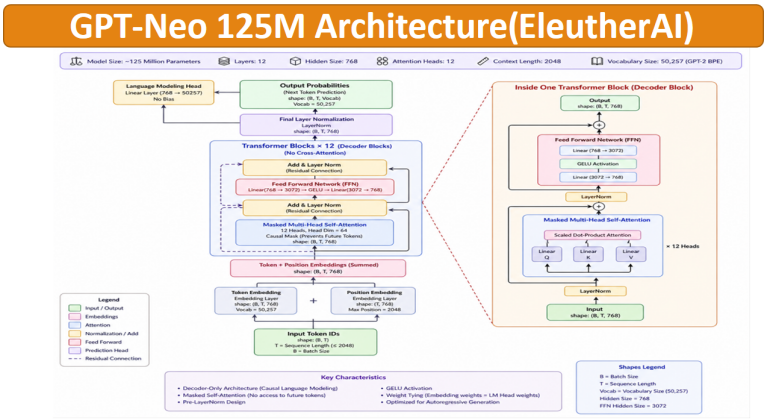

Figure 3.5: GPT-Neo model with LoRA fine-tuning

3.7 Training Configuration

Transformer-based causal language modeling used to train the models. The transformer gets an input tokens sequence during training and makes an autoregressive prediction of subsequent token. During training, the following elements were utilized:

- AdamW Optimizer
- Cross-Entropy Loss Function
- Autoregressive Causal Language Modeling
- Batch-based Training
- GPU-based Fine-Tuning

AdamW optimizer is chosen due of its effective weight regularization capabilities robust convergence characteristics. Because it reduces overfitting and enhances optimization efficiency, it is frequently employed in transformer training. discrepancy between expected probability distributions actual target tokens is measured by cross-entropy loss. Reducing this loss across several epochs is the goal of training.

3.8 Evaluation Metrics

3.8.1 Loss

Because it quantifies prediction error, loss is one of most significant assessment metrics in language modeling deep learning. The transformer model creates probability distributions for potential next tokens in next-word prediction problems. actual target token and their estimated probabilities are compared by loss function. During training, a decreasing loss number means that model's predictions are getting more accurate. Because it efficiently quantifies the difference between expected and actual probability distributions, cross-entropy loss is frequently utilized in language modeling. For example:

Input: भारत एक

Correct Target:

महान

The loss is reduced if model gives right token a high probability. On other hand, the loss value rises if right token has a low probability. Validation loss training loss are both crucial assessment metrics. While validation loss assesses model's capacity for generalization on untested samples, training loss gauges model's performance on training data.

3.8.2 Perplexity

Relationship between loss and perplexity is given by:

$$Perplexity = e^{Loss}$$

Because it gauges how well a model anticipates the next token in a sequence, perplexity is one of most used evaluation metrics in language modeling [4]. way to think of perplexity is as a gauge of prediction uncertainty. Reduced perplexity values show that model predicts following word with greater confidence and contextual accuracy. Poorer prediction quality and a lack of contextual awareness are indicated by higher confusion levels. For example:

- Perplexity = 2 : Highly confident predictions
- Perplexity = 10 : Moderate uncertainty
- Perplexity = 50 : Poor prediction capability

Loss perplexity are exponentially related. Therefore, even small reductions in loss may lead to substantial improvements in perplexity scores.

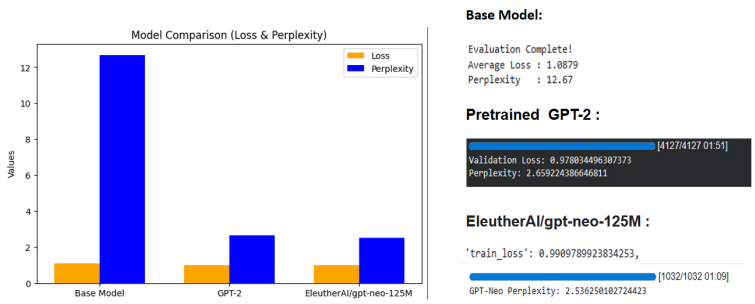

Figure 3.6: Model performance comparison based on loss and perplexity

3.9 Experimental Results

Table 3.1: Model Performance Comparison

Model	Validation Loss	Perplexity
GPT-2 (Scratch)	1.0879	12.67
GPT-2 + LoRA	0.9780	2.65
GPT-Neo + LoRA	0.9909	2.53

The experimental results show models trained completely from scratch are considerably outperformed by pretrained transformer models fine-tuned using LoRA. Because the GPT-2 scratch model only used existing Hindi dataset to understand contextual associations, it resulted in increased validation loss and confusion. Its capacity to successfully generalize was hampered by the small sample size and computational limitations. contextual information obtained following extensive pretraining helped GPT-2 with LoRA fine-tuning. model was able to effectively adjust to Hindi grammatical structures and lexical patterns by fine-tuning on Hindi text. GPT-Neo with LoRA performed the best in terms of perplexity out of all the models that were examined. This suggests that the quality of Hindi next-word prediction may be significantly increased by combining bigger pretrained transformer topologies with parameter-efficient adaptation strategies.

3.10 Inference (Next Word Prediction)

Example:

- Input: भारत एक
- Output: महान

The process of making predictions using the learned transformer model following training is known as inference. The transformer model uses contextual patterns discovered during training to anticipate the most likely next token when a user inputs a Hindi sentence or phrase. For example:

मुझे हिंदी

Possible prediction:

सीखना

Predictions are generated autoregressively, meaning that each predicted token is appended to the input sequence for subsequent prediction steps. Inference quality depends heavily on dataset quality, contextual learning capability, tokenizer efficiency, and transformer architecture design.

Figure 3.7: Example of next-word prediction using trained model

3.11 Deployment using Streamlit

Model was included into a Streamlit-based online interface for real-time Hindi next-word prediction following training and assessment. Users may enter Hindi text sequences into deployment interface and interactively obtain the following words that anticipated. This illustrates the created transformer-based language modeling system's usefulness. For interactive NLP applications and quick prototyping, Streamlit offers a lightweight and intuitive deployment architecture. Following steps are part of the deployment pipeline:

- Loading trained tokenizer
- Loading the LoRA fine-tuned transformer model
- Accepting user text input
- Token generation
- Prediction decoding
- Real-time output visualization

CHAPTER 4

EXPERIMENTS AND RESULTS

4.1 Introduction

Standard language modeling measures as like validated loss,perplexity [4] were used of assess the models performance on the bespoke Hindi dataset presented in the preceding chapter. Primary goal of research is determined good various transformer topologies can anticipate the multiple-next significant word in a Hindi phrase while preserving contextual consistency.
Experimental assessments are crucial for natural language production tasks because ,showed how well model's can learn grammatical structures,semantic patterns,contextual relationships from textual input.Since contextual comprehension plays a major role in next-word prediction,quantitative metrics analysis of transformer model activity is required.

- GPT-2 trained from scratch
- GPT-2 with LoRA fine-tuning
- GPT-Neo-125M with LoRA fine-tuning

4.2 Experimental Setup

All the tests are carries as out utilizing a uniform training pipeline and preprocessing technique to guarantee equitable comparison across various transformer topologies.
Main goal of autoregressive causal modeling, which were used to train the transformer models,is to predict the subsequent token from previously observed tokens in the sequence.
The following phases make up the entire experimental pipeline:

- Hindi dataset preprocessing
- SentencePiece tokenizer training
- Dataset tokenization
- Transformer model initialization

- Fine-tuning using LoRA
- Validation loss and inference testing

GPU-based training were used to the studies to speed up computation and increase transformer optimization effectiveness. HuggingFace Transformers and PEFT libraries were used in the training process to construct transformer designs and LoRA-based adaptability.Main deep learning framework for gradient optimization and LLM's model training was PyTorch.
The following experimental components were used:

- Framework: PyTorch
- Transformer Library: HuggingFace Transformers
- Fine-Tuning Library: PEFT (Parameter-Efficient Fine-Tuning)
- Tokenizer: SentencePiece with BPE
- Optimizer: AdamW
- Loss Function: Cross-Entropy Loss

4.3 Training Configuration

All transformer architectures were trained using similar hyperparameter settings to maintain consistency during experimental comparison.
The major training hyperparameters were summarized below:

Table 4.1: Training Hyperparameters

Parameter	**Value**
Batch Size	8
Learning Rate	2×10^{-5}
Optimizer	AdamW
Sequence Length	128
Epochs	5
Vocabulary Size	8000
Tokenizer	SentencePiece BPE

The sequence length is 128 tokens were chosen to balance contextual learning capability and GPU memory constraints.Longer sequences generally improve contextual understanding but substantially increase computational cost.

Because of excellent optimization performance of the transformer-based systems, the AdamW optimizer was chosen. AdamW's weight decay regularization enhances generalization capacity while lowering overfitting.
Because it efficiently quantifies prediction divergence between generated probabilities are distributed and also,target tokens, Cross-Entropy Loss was chosen as the main optimization goal.

4.4 Evaluation Metrics

Analyzing efficacy to the LLM's models requires the use of evaluation measures. Metrics like validation loss,perplexity are frequently used of assess contextual knowledge,prediction quality in autoregressive language generation systems.
These measures aid in assessing how good a model forecasts the subsequent token from a series are of tokens that have already been observed.Stronger language modeling ability and improved contextual learning were often indicated by lower metric values.

4.4.1 Loss

Model's prediction error during next-token generations are represented as loss.Stronger contextual knowledge,improved predictions accuracy were often indicated by lower loss levels.
Cross-Entropy Loss is frequently employed in deep learning-based language models because it efficiently quantifies the discrepancy between the actual target token,projected probability distribution.
Transformer model's creates probability for every potential token in the vocabulary during training.Prediction error is determined by loss function by comparing these predicted probabilities with a appropriate target token.
For example:

Input Sentence: भारत एक

Correct Next Word:

महान

Loss value decreases if the LLM's model assigns a high probability to the token महान.Other hand,loss value rises if unrelated terms are given greater probability.
Overfitting can be also identified via loss curves.The model may have started overfitting the training dataset if the validation error begin of rise while training loss continues to decline.

4.4.2 Perplexity

Perplexity is mathematically expressed as:

$$Perplexity = e^{Loss} \tag{4.1}$$

One common evaluation metric used in language modeling to assess how well a probability model predicts word sequences is perplexity [4].Stronger language comprehension and enhanced prediction ability are indicated by lower perplexity levels.

Perplexity might be seen as a measurement of prediction uncertainty in next-word prediction systems,and shows the model's level of confidence when choosing the subsequent token from the vocabulary.

The algorithm predicts the following word with more confidence and contextual relevance when the perplexity score is lower.Higher confusion,other hand,suggests poorer contextual comprehension and increased prediction uncertainty.

For example:

- Perplexity = 2 indicates highly confident predictions.
- Perplexity = 10 indicates moderate prediction uncertainty.
- Perplexity = 50 indicates weak contextual understanding.

Because Hindi has numerous inflectional forms, sophisticated grammatical structures, and contextual variables,perplexity are especially crucial for Hindi language modeling. Consequently, decreasing confusion is sign of enhanced contextual learning and a deeper comprehension Hindi language patterns.

The exponential dependency of bewilderment on loss is another important characteristic. A discernible very improvement in perplexity scores can result from even a slight reduction of loss.

Figure 4.1: Training and validation loss comparison across models

The convergence behavior is various transformer topologies during training is showed in Figure 4.1.When compared to models that are completely trained from scratch,these are evident that pretrained models that are refined using LoRA converge more quickly,reduced validation loss.

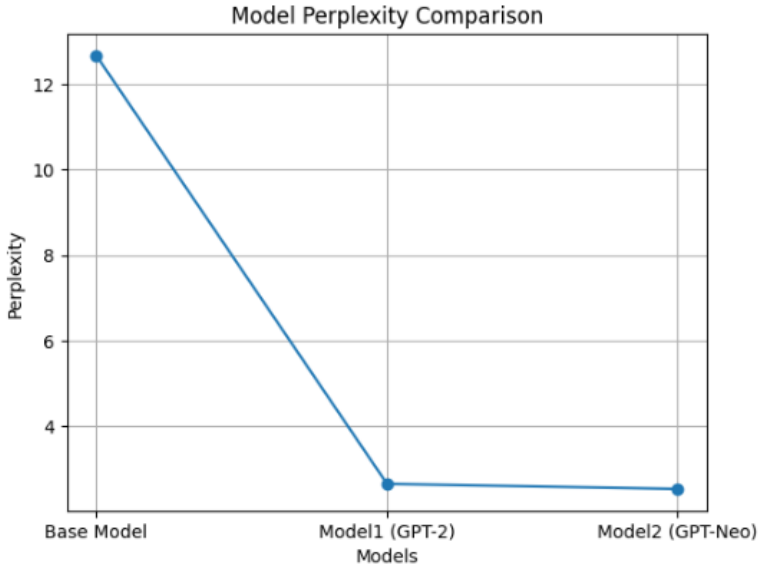

Figure 4.2: Perplexity comparison across different models

Figure 4.2 presents perplexity comparison among of the evaluated transformer architectures.Lower perplexity values correspond to stronger contextual learning and improved next-word prediction performance.

4.5 LLM's Model Comparison

Three transformer architectures were evaluated during experimentation:

- GPT-2 (trained from scratch) [3]
- GPT-2 (Pretrained + LoRA fine-tuning) [8]
- GPT-Neo-125M (Pretrained + LoRA fine-tuning) [5]

Analyzing the pretraining,parameter-efficient adaptation affect Hindi next-word prediction performance is the goal of comparing various architectures.
The first model, GPT-2, was trained exclusively,bespoke Hindi corpus after being started with random weights.In this configuration, the available training data were used to immediately learn all contextual connections and linguistic patterns.
The second design used LoRA adaptation in conjunction with pretrained GPT-2 weights. GPT-2 could better adapt Hindi language patterns through fine-tuning since it already has contextual information acquired from large text corpora.
GPT-Neo-125M with LoRA fine-tuning was employed in the third architecture. Semantic comprehension and prediction quality may be enhanced by GPT-Neo's better pretrained contextual representations and somewhat bigger transformer architecture.

Table 4.2: Model Performance Comparison

Model	**Validation Loss**	**Perplexity**
GPT-2 (Scratch)	1.0879	12.67
GPT-2 + LoRA	0.9780	2.65
GPT-Neo + LoRA	0.9909	2.53

This is evident from the experimental comparison that the architecture taught from scratch is inferior than pretrained transformer models.
Because it is used only for the available Hindi dataset for contextual learning, the GPT-2 scratch model had the maximum perplexity. The model has trouble efficiently generalizing across a variety of phrase forms because of the small dataset size and lack of prior pretrained information.
Pretrained transformer representations greatly enhance contextual awareness for Hindi language modeling tasks, as seen by the large decrease in validation loss that GPT-2 with LoRA fine-tuning accomplished.

GPT-Neo-125M with LoRA has the highest perplexity score of all the architectures that were assessed. This finding implies that next-word prediction quality may be significantly increased by combining bigger pretrained transformer designs with parameter-efficient fine-tuning.

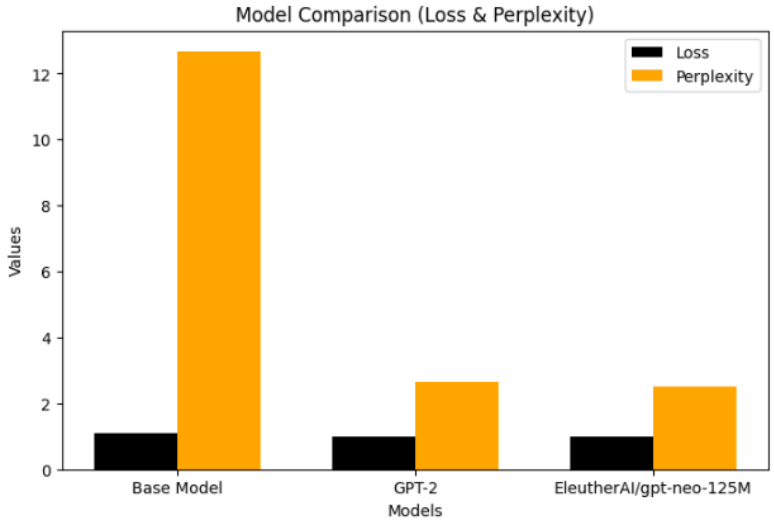

Figure 4.3: Comparison of model performance based on loss and perplexity

The results of the experiments show that pretrained transformer models that are modified using LoRA perform noticeably better than models that are trained from scratch. GPT-Neo had the highest perplexity performance, whereas GPT-2 with LoRA had the lowest validation loss.
The findings confirm that parameter-efficient fine-tuning methods, like LoRA, are beneficial for multilingual NLP applications. Compared to complete transformer retraining, LoRA allows for effective adaptation with fewer trainable parameters and less computing cost.

4.6 Inference Results

The trained transformer models were tested on multiple Hindi text sequences to evaluate next-word prediction capability.
Example:

- Input: भारत एक
- Predicted Output: महान

After training is finished, the model uses the contextual relationships it has learnt to predict the most likely next token from the input sequence.
Contextual comprehension and semantic fluency were assessed in a number of Hindi sentence samples.

Example 1:

Input: मुझे हिंदी

Predicted Output:

सीखना

Example 2:

Input: आज मौसम

Predicted Output:

अच्छा

These examples indicate that the transformer architectures successfully learned meaningful contextual patterns and generated semantically relevant Hindi predictions.
The autoregressive behavior of GPT-style models enables sequential token generation, where each predicted token becomes part of the input for future prediction steps.

Figure 4.4: Example of Hindi next-word prediction

The generated predictions demonstrate that the trained models are capable of producing meaningful and contextually appropriate Hindi words, reflecting their ability to capture linguistic dependencies [2].
The inference outcomes further show that pretrained transformer architectures can adapt successfully to Hindi text generation tasks even though the original pretraining was performed predominantly on English-language corpora.

4.7 Qualitative Analysis of Generated Predictions

Beyond numerical evaluation metrics, qualitative analysis is also important for understanding the fluency and contextual relevance of generated Hindi text.
The generated outputs indicate that the transformer models learned several important linguistic properties of Hindi, including:

- Contextual word relationships
- Sentence continuity
- Basic grammatical agreement
- Common Hindi phrase structures
- Semantic consistency

In many cases, the generated next words were contextually meaningful and grammatically appropriate. This demonstrates that the transformer models successfully captured important contextual dependencies from the training corpus.
However, prediction quality occasionally varied for highly complex or ambiguous sentences. Since Hindi contains flexible sentence structures and multiple grammatical forms, certain contextual relationships remain difficult to model accurately using relatively small transformer architectures.
The qualitative analysis also suggests that larger datasets and more advanced transformer architectures may further improve semantic fluency and long-range contextual understanding.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 Conclusion

The motivation behind this research emerged from two major challenges in Hindi Natural Language Processing. First, compared to English, Hindi still lacks highly optimized and widely available language models. Second, training large transformer architectures generally requires significant computational resources, which may not always be accessible in academic or low-resource environments. The collected text was carefully cleaned, normalized, and prepared for causal language modeling before applying SentencePiece tokenization [6].
This work mainly focused on the following objectives:

- Building an effective Hindi next-word prediction system
- Reducing computational complexity during transformer fine-tuning

Hindi is a morphologically rich language that contains complex grammar, contextual variations, and multiple word forms. Since most pretrained transformer models are trained predominantly on English-centric corpora, their performance on Hindi NLP tasks is often limited. Therefore, this research explored how transformer-based architectures could be adapted efficiently for Hindi language modeling.
The overall methodology included data collection, preprocessing, tokenizer training, transformer model selection, fine-tuning, evaluation, inference generation, and deployment. A custom Hindi corpus containing nearly 167850 text samples was prepared from multiple textual sources. During preprocessing, duplicate entries, noisy symbols, unwanted characters, and formatting inconsistencies were removed to improve dataset quality and consistency.
SentencePiece-based subword tokenization [6] was selected because of its effectiveness in handling morphologically rich languages such as Hindi. The tokenizer converted Hindi text into meaningful subword representations, helping reduce out-of-vocabulary issues while improving vocabulary handling during prediction.
Three different transformer model configurations were evaluated in this work:

- GPT-2 trained from scratch [3]
- GPT-2 with LoRA fine-tuning [8]
- GPT-Neo-125M with LoRA fine-tuning [5]

Additionally, parameter-efficient fine-tuning approaches such as QLoRA (Quantized Low-Rank Adaptation) [9] were explored to reduce memory usage and computational cost during training.

The experimental analysis clearly showed that pretrained transformer architectures perform considerably better than models trained entirely from scratch. The GPT-2 scratch model required significantly more effort to learn Hindi contextual relationships because all parameters were initialized randomly and trained solely on the custom dataset.

The experimental results demonstrated that pretrained models achieved better contextual understanding and prediction capability. The GPT-2 model trained from scratch obtained a validation loss of 1.0879 and perplexity of 12.67, indicating weaker language modeling performance. In comparison, GPT-2 with LoRA fine-tuning achieved a validation loss of 0.9780 and perplexity of 2.65, showing substantial improvement in contextual prediction quality.

Similarly, GPT-Neo-125M with LoRA achieved the best perplexity score of 2.53 along with a validation loss of 0.9909. These findings suggest that larger pretrained transformer architectures combined with parameter-efficient adaptation techniques can effectively improve Hindi language modeling performance.

The inference results further confirmed that the trained models could generate meaningful and contextually relevant Hindi predictions.

For example:

Input: भारत एक

Predicted Output:

महान

Similarly:

Input: मुझे हिंदी

Predicted Output:

सीखना

5.2 Limitations

Although the proposed system achieved promising results, several limitations still exist.

- **Dataset limitation**: Although the dataset is relatively large, it may not fully represent all Hindi language domains.

  The collected corpus mainly contains educational content, articles, books, and publicly available textual resources. However, Hindi language usage varies significantly across domains such as social media conversations, legal documents, poetry, regional dialects, and technical writing. Limited domain diversity may reduce the model's generalization capability on unseen sentence structures.

- **Limited model size**: The transformer models used in research (GPT-2 and GPT-Neo-125M) are relatively small compared with modern large-scale LLMs [4].

  Modern transformer architectures such as GPT-3, LLaMA, Mistral, and GPT-NeoX contain billions of parameters and possess stronger contextual representation capabilities. Due to hardware limitations, this work relied on comparatively smaller transformer architectures.

- **Short context length**: The sequence length was limited to 128 tokens during training.

  Transformer models generally perform better when longer contextual sequences are available. Because of this limitation, the trained models may struggle to capture very long-range contextual dependencies in lengthy Hindi paragraphs.

- **Training time constraints**: Due to GPU and memory limitations, pretrained models were trained for a limited number of epochs.

  Additional training epochs and larger-scale fine-tuning could potentially improve contextual understanding and prediction quality further.

- **Limited evaluation metrics**: The evaluation mainly relied on validation loss and perplexity.

  Although these are standard language modeling metrics, they do not completely measure semantic correctness, grammatical fluency, or human readability. Human evaluation could provide a more comprehensive assessment of generated Hindi text quality.

- **Limited multilingual capability**: The proposed system currently focuses only on Hindi language prediction.

  Extending the framework to multilingual Indian language modeling would require additional datasets, multilingual tokenization strategies, and larger-scale training approaches.

5.3 Future Work

There are several possible directions for extending this research in the future.

- **Using larger transformer models**: Future work may investigate larger architectures such as GPT-NeoX, LLaMA, or Mistral for improved performance.

  Larger transformer architectures generally possess stronger semantic understanding and better contextual representation capability, which may further improve Hindi text generation quality.

- **Multilingual extension**: The framework may be extended to support additional Indian languages like Tamil, Telugu, Bengali, Marathi, and Punjabi.

  Expanding the system toward multilingual Indian language modeling may significantly improve regional NLP accessibility and multilingual AI research.

- **Improved evaluation metrics**: Additional evaluation measures such as BLEU, ROUGE, semantic similarity scores, and human evaluation may provide deeper insights into generated text quality.

  Human evaluation may help analyze grammatical fluency, readability, semantic relevance, and contextual correctness more effectively.

- **Long-context language modeling**: Increasing sequence length during training may help models capture paragraph-level dependencies and long-range semantic relationships more effectively.

- **Real-time NLP applications**: The developed system may be integrated into practical applications such as:

  - Hindi predictive keyboards
  - Educational writing assistants
  - Conversational AI systems

  - Smart autocomplete systems
  - Hindi virtual assistants
  - AI-powered text generation tools

- **Advanced fine-tuning approaches**: Future studies may explore prefix tuning [10], prompt tuning, instruction tuning, and adapter tuning to further reduce computational requirements while improving performance.

- **Integration with RAG**:

  Retrieval-Augmented Generation techniques may improve factual consistency and contextual relevance by retrieving external Hindi knowledge during inference.

- **Quantized deployment for edge devices**:

  Quantization and model compression techniques may enable deployment of Hindi language models on low-resource systems and mobile devices.

- **Conversational Hindi AI systems**:

  Future research may also focus on dialogue generation systems and conversational Hindi assistants capable of maintaining long-term contextual interaction.

Bibliography

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[2] Ankit Sharma and Komal Gupta. Next word prediction in hindi using deep learning techniques. In *2019 International Conference on Intelligent Computing and Control Systems (ICCS)*, pages 1302–1306. IEEE, 2019.

[3] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8):9, 2019.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

[5] Sid Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. Gpt-neo: Large scale autoregressive language modeling with mesh-tensorflow. *Zenodo*, 2021.

[6] Taku Kudo and John Richardson. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 66–71, 2018.

[7] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 1715–1725, 2016.

[8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[9] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. *arXiv preprint arXiv:2305.14314*, 2023.

[10] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. *arXiv preprint arXiv:2101.00190*, 2021.